iLink 使用者实操手册

> ***读者***：*使用 iLink 进行日常开发的工程师* ***前提***：*项目已完成 Bootstrap（`project-context.md` 和入口文件已就位）* ***版本***：*v1.2.00*

目录

- 1. 三分钟上手
- 2. 写需求定义——你最重要的工作
- 3. 流水线操作详解
- 4. 审核设计——Human-Gate 实操
- 5. 修订 STAGING 文档——ilink-refine 实操
- 6. 回流与熔断——出了问题怎么办
- 7. 维护 project-context.md
- 8. 多 Story 并行开发
- 9. 常见问题排查
- 10. 反模式——千万别这样做
- 附录 A：命令速查表
- 附录 B：文件地图
- 附录 C：状态流转图

---

1. 三分钟上手

1.1 最小操作流程

```
# ① 创建 Story（对应你的 Jira 单号）
/ilink-init kcia-1520

# ② 编辑需求定义（这是你唯一需要手写的文件）
#    打开 iLink-doc/kcia-1520/kcia-1520-需求定义.md，填写内容

# ③ 启动 AI 流水线
/ilink-pm kcia-1520          # AI 分析需求，输出业务合同
/ilink-design kcia-1520      # AI 做技术设计，输出文件清单

# ④ 你审核设计（这是你最重要的审核点）
#    阅读 iLink-doc/kcia-1520/kcia-1520-design.master.md
#    重点看 [TASK_ALLOCATION] 和 [DESIGN_DECISIONS]
ilink-approve kcia-1520      # 审核通过，推进到编码

# ⑤ AI 编码 + 审查
/ilink-coder kcia-1520       # AI 写代码
/ilink-qa kcia-1520          # AI 审查代码

# ⑥ 你做最终确认后提交
git add .
git commit -m "kcia-1520: 功能描述"
```

整个过程中你只做三件事：写需求、审设计、最终提交。

1.2 遇到 STAGING 时

如果 AI 在某个阶段输出了 `STAGING`（表示有不确定项），你有四种处理方式：

```
# 方式 A（推荐）：修订对话，逐条确认
/ilink-refine kcia-1520      # AI 列出所有 [待确认] 项，你逐条决策
ilink-approve kcia-1520      # 确认完毕后推进

# 方式 B：直接编辑文档
# 手动修改 STAGING 文档中的阻塞项
ilink-approve kcia-1520

# 方式 C：修改上游重跑
# 修改需求定义，重新执行对应角色命令

# 方式 D：直接推进（承担风险）
ilink-approve kcia-1520      # 未解决的 [待确认] 项原样传入下游
```

1.3 查看进度

随时可以查看 Story 的当前状态：

```
ilink-status              # 查看所有 Story 概览
ilink-status kcia-1520    # 查看指定 Story 详情
```

---

2. 写需求定义——你最重要的工作

2.1 为什么需求定义这么重要

需求定义是整条流水线的**唯一人类输入**。AI 的 PM → Designer → Coder → QA 全部基于你写的需求定义展开。需求定义的质量直接决定最终代码的质量。

一个写了 10 分钟的需求定义，能省你 2 小时的返工。

2.2 需求定义模板

执行 `/ilink-init <story>` 后，会在 `iLink-doc/<story>/` 下生成模板：

```
# <story> — 需求定义

## 1. 功能描述
（用 1-3 句话说清楚要做什么）

## 2. 功能范围
- In Scope:
  - （明确要做的事）
- Out of Scope:
  - （明确不做的事）

## 3. 验收标准
- AC-01: （可验证的条件）
- AC-02: （可验证的条件）

## 4. 约束备注
（已知的技术约束、兼容性要求、特殊注意事项）

## 5. 假设与风险
（你已知的假设、不确定的事项、潜在风险）
```

2.3 写好每个章节

§ 1 功能描述

目标：让一个不了解背景的人在 10 秒内知道你要做什么。

```
❌ 差：优化登录模块
✅ 好：新增用户登录失败统计接口，供运维监控使用，返回最近 7 天每天的失败次数
```

§ 2 功能范围

这是最关键的章节。 AI 设计和编码的边界完全由这里决定。

```
❌ 差：
- In Scope: 登录失败统计

✅ 好：
- In Scope:
  - 新增 REST 接口 /api/cia/login-fail-stat，返回最近 N 天的失败统计
  - 在 fs_cia_login_log 表上增加索引优化查询性能
  - 支持按操作员维度和全局维度两种查询方式
- Out of Scope:
  - 登录失败实时告警（属于 kcia-1521）
  - 修改现有登录逻辑
  - 前端页面展示
```

"Out of Scope" 同样重要。 它防止 AI 多做事。

§ 3 验收标准

每条验收标准 MUST 是**可验证的**——读完后能明确判断"做到了"还是"没做到"。

```
❌ 差：
- AC-01: 接口性能好

✅ 好：
- AC-01: GET /api/cia/login-fail-stat?days=7 返回 JSON 数组，
         格式 [{date: "2026-04-01", count: 15}, ...]
- AC-02: 百万级 login_log 数据下，接口响应时间 P99 < 200ms
- AC-03: 无登录记录时返回空数组 []，不返回错误
```

QA 阶段会逐条核对验收标准。 写得越精确，QA 越有效。

§ 4 约束备注

把你知道但 AI 可能不知道的事写在这里：

```
## 4. 约束备注
- 必须兼容现有 fs_cia_login_log 表结构，不能加字段
- 接口需注册到 BEX 调度器，功能码前缀用 920
- 该接口不需要登录态验证（属于管理端接口，走 mng 服务）
- 参考现有接口：FsOperatorQueryHandler（查询类接口的标准实现模式）
```

"参考现有接口"特别有用。 AI 会去读那个接口的代码，模仿它的实现模式。

2.4 需求定义自查清单

写完后用这个清单检查一遍：

- [ ] 功能描述能让不了解背景的人看懂
- [ ] In Scope 列出了所有要做的事
- [ ] Out of Scope 列出了容易被误解为要做但实际不做的事
- [ ] 每条验收标准都能明确判断"做到 / 没做到"
- [ ] 约束备注中写了 AI 不可能从代码中推断出的信息
- [ ] 如果有参考实现，已经写明了文件路径或类名

---

3. 流水线操作详解

3.1 /ilink-init — 创建 Story

```
/ilink-init kcia-1520
```

执行效果：

- 创建 `iLink-doc/kcia-1520/` 目录
- 生成 `kcia-1520-需求定义.md` 模板

执行后你要做的：编辑需求定义，填写完整内容。

3.2 /ilink-pm — AI 需求分析

```
/ilink-pm kcia-1520
```

AI 读取：需求定义.md + project-context.md **AI 输出**：`kcia-1520-pm.master.md`（业务合同）

业务合同包含：

- A 层：功能摘要
- B 层：范围契约、硬约束、验收标准契约、假设与风险
- C 层：给 Designer 的调度通知

你通常不需要审核 PM 输出。 Designer 在做设计时会天然审查 PM 的理解是否正确。 但如果你想看一眼确认 AI 是否理解了你的意图，可以快速扫一下 B1（范围契约）。

可能的状态：

- `PENDING_DESIGNER` — 正常，进入下一步
- `STAGING` — AI 认为需求有风险或不明确。有两种处理方式：
  - 执行 `/ilink-refine kcia-1520` 逐条确认 `[待确认]` 项（推荐）
  - 执行 `ilink-approve kcia-1520` 直接推进（承担风险）

3.3 /ilink-design — AI 技术设计

```
/ilink-design kcia-1520
```

AI 读取：pm.master.md + project-context.md + 源码 **AI 输出**：`kcia-1520-design.master.md`（技术设计）

设计文档包含：

- 设计概述和系统逻辑分析
- 技术设计方案（类设计、方法签名、接口设计）
- `[DESIGN_DECISIONS]` — AI 做的关键技术决策
- `[TASK_ALLOCATION]` — **文件级任务清单**（最重要）

设计完成后状态为 `STAGING`，等待你审核（见第 4 章）。

3.4 ilink-approve — 审核通过

```
ilink-approve kcia-1520
```

前置条件：存在 STAGING 状态的文档 **执行效果**：

- design.master.md STAGING → `PENDING_CODER`
- pm.master.md STAGING → `PENDING_DESIGNER`

3.5 /ilink-coder — AI 编码

```
/ilink-coder kcia-1520
```

AI 读取：design.master.md + project-context.md + 源码文件 **AI 输出**：修改/新增的源码文件 + `kcia-1520-code.master.md`

重要：Coder 直接写文件到磁盘。执行完后检查 `git diff` 查看实际变更。

code.master.md 包含：

- 变更清单（改了哪些文件、怎么改的）
- `[REVIEW_HANDOFF]` — 给 QA 的审查要点
- `[DEVIATIONS]` — 与设计的偏差说明（如果有的话）

3.6 /ilink-qa — AI 代码审查

```
/ilink-qa kcia-1520
```

AI 读取：code.master.md + 源码 + design.master.md + pm.master.md + project-context.md **AI 输出**：`kcia-1520-review.master.md`（审查报告）

可能的结论：

状态	含义	你要做什么
COMPLETED	审查通过	你做最终审核后 `git commit`
FAIL_BACK_TO_CODER	代码有问题，Coder 可修	执行 `/ilink-coder kcia-1520` 回流修复
STAGING	上游问题（设计或需求有缺陷）	看 `[UPSTREAM_BLOCKERS]`，可执行 `/ilink-refine` 讨论根因

---

4. 审核设计——Human-Gate 实操

这是你在整个流水线中最重要的审核节点。 设计方向对了，后面基本不出大问题；设计方向错了，后面全部返工。

4.1 打开设计文档

```
# 查看设计文档
cat iLink-doc/kcia-1520/kcia-1520-design.master.md
```

4.2 审核 `[TASK_ALLOCATION]`（必看）

这是 Coder 的"白名单"。**Coder 只会修改这里列出的文件。**

```
## [TASK_ALLOCATION]

| 任务 | 修改文件 | 修改类型 | 说明 |
|------|---------|---------|------|
| T-01 | .../FsLoginFailStatHandler.java | 新增 | 登录失败统计处理器 |
| T-02 | .../mapper/FsLoginFailStatMapper.xml | 新增 | MyBatis 查询映射 |
| T-03 | .../mapper/FsLoginFailStatMapper.java | 新增 | Mapper 接口 |
| T-04 | server/fs-cia-mng/.../application.yml | 修改 | 注册功能码 |
```

你要检查：

1. **该有的文件都列了吗？**
   - 比如新增接口通常需要：Handler + Mapper + XML + 注册配置
   - 如果少了 Mapper 接口，说明设计可能遗漏了数据层
2. **不该有的文件出现了吗？**
   - 比如你说了 Out of Scope 的某个模块，但设计里列了那个模块的文件
3. **修改类型合理吗？**
   - "修改"了一个你觉得不该碰的文件 → 要问为什么
   - "新增"的文件路径是否在正确的包下

4.3 审核 `[DESIGN_DECISIONS]`（必看）

```
## [DESIGN_DECISIONS]

- DD-01: 选择在 fs-cia-mng 服务（SpringMVC）而非 fs-cia-lis 服务（WebFlux）中实现，
         因为统计接口是管理端功能，且 mng 已有类似的查询接口模式。

- DD-02: 使用 MultiJdbcTemplate 直接执行 SQL 而非 MyBatis Plus，
         因为统计查询涉及 GROUP BY + 日期函数，ORM 反而增加复杂度。
```

你要检查：

- 每个决策的理由是否成立
- 是否和你的预期一致
- 如果你不同意某个决策 → **现在修改成本最低**

4.4 快速审核 vs 深度审核

简单 Story（改配置、加简单接口）：看 `[TASK_ALLOCATION]` 的文件列表，确认没问题就 approve，1–2 分钟。

复杂 Story（新增模块、涉及多表关联、有性能要求）：仔细看设计概述、类设计、`[DESIGN_DECISIONS]`，可能需要 10–15 分钟。

4.5 审核不通过怎么办

如果设计有问题，**不要 approve**。有三种处理方式：

方式 A（推荐）：执行 ilink-refine 修订

```
/ilink-refine kcia-1520
# AI 会列出设计中的关键决策和待确认项
# 你逐条告诉 AI 哪些要改，AI 在原文档上就地修改
# 修订完成后执行 ilink-approve
```

方式 B：你手动修改设计文档 直接编辑 `design.master.md`，改完后执行 `ilink-approve kcia-1520`。

方式 C：修改上游重跑 如果根因在 PM 的需求理解，修改需求定义后重新执行 `/ilink-pm` 和 `/ilink-design`。

---

5. 修订 STAGING 文档——ilink-refine 实操

5.1 什么是 ilink-refine

`ilink-refine` 是 iLink 的**修订对话协议**。当文档处于 STAGING 状态时，你可以通过它与 AI 逐条确认阻塞项，而**不需要重新生成整个文档**。

5.2 什么时候用

场景	是否用 refine
PM 输出 STAGING，有［待确认］项	✅ 用 refine 逐条确认
Designer 输出 STAGING，你想微调设计决策	✅ 用 refine 讨论修改
QA 输出 STAGING（上游根因），需讨论处理路径	✅ 用 refine 讨论根因
你已经知道答案，想直接推进	❌ 直接 `ilink-approve`
需要大幅调整方向	❌ 修改上游重跑更合适

5.3 操作流程

```
# 1. 执行 refine
/ilink-refine kcia-1520

# 2. AI 会:
#    - 找到当前 STAGING 的文档（优先级: review > design > pm）
#    - 列出所有阻塞项（[待确认]、[UPSTREAM_BLOCKERS] 等）
#    - 逐条向你确认

# 3. 你逐条给出决策:
#    "AR-03 的假设成立，基础平台组确认本季度不升级 KDEncode"
#    → AI 记录为 [已确认 2026-04-14: 基础平台组确认本季度不升级 KDEncode]

# 4. 所有阻塞项处理完后
ilink-approve kcia-1520      # 正式推进状态
```

5.4 refine 的关键规则

- **修订不是重生成**：AI 只修改被讨论的条目，保留文档其余内容不变
- **状态不变**：refine 完成后文档仍是 STAGING，状态推进由 `ilink-approve` 负责
- `[已确认]` **格式**：`[已确认 YYYY-MM-DD: <人类给出的决策依据>]`
- `[已确认]` **的效力**：下游角色 MUST 将其视为绑定约束，不再质疑

5.5 各角色文档的 refine 重点

文档	阻塞项位置	refine 目标
pm.master.md	B5［待确认］+ C1 通知项	逐条确认假设与风险
design.master.md	[DESIGN_DECISIONS] 风险应对表	确认技术方向和设计假设
review.master.md（STAGING）	[UPSTREAM_BLOCKERS]	讨论上游根因，明确修改路径

---

6. 回流与熔断——出了问题怎么办

6.1 正常回流

QA 审查发现代码有问题，判定根因在 Coder 时：

```
QA 输出 Status: FAIL_BACK_TO_CODER
    ↓
你执行: /ilink-coder kcia-1520    ← Coder 读取 QA 的 [FIX_REQUESTS]
    ↓
Coder 修复后，你再执行: /ilink-qa kcia-1520
```

Coder 在回流时只修复 QA 指出的问题，不会做额外修改。

6.2 回流计数与熔断

每次回流，`.retry_count` 文件中的计数 +1。

```
第 1 次回流 → 正常，Coder 修复
第 2 次回流 → 正常，Coder 修复
第 3 次回流 → 触发熔断警告 ⚠️
```

熔断意味着：AI 自动修复了 3 次还没通过 QA。这通常说明：

- 需求或设计本身有问题
- 或者修复方向偏离了

你需要做的：

1. 阅读最新的 `review.master.md`，看 QA 指出的问题
2. 阅读 `code.master.md` 中的 `[FIX_RESPONSE]`，看 Coder 尝试了什么
3. 判断是设计问题还是实现问题
4. 如果是设计问题 → 回到 Designer 阶段重新设计
5. 如果是实现问题 → 你手动介入修复代码

6.3 上游问题（STAGING）

QA 判定问题根因不在 Coder，而在设计或需求时：

```
QA 输出 Status: STAGING
    ↓
查看 review.master.md 中的 [UPSTREAM_BLOCKERS]
    ↓
执行 /ilink-refine kcia-1520（与 AI 讨论根因，明确修改路径）
    ↓
根据讨论结果决定：
├── 需求问题 → 修改需求定义，从 /ilink-pm 重新开始
└── 设计问题 → 修改设计或重新执行 /ilink-design
```

---

7. 维护 project-context.md

7.1 什么是 project-context.md

这是你项目的**知识库**，位于项目根目录。所有 AI 角色在执行任何任务前都会读它。

它包含：

- 项目概述和技术栈

- 技术约束（Java 版本、框架约束等）
- 模块职责和依赖关系
- 架构原则
- 构建命令
- 特殊说明

7.2 什么时候更新

发生了什么	是否更新	更新哪个章节
新增了一个子模块	✅	§3 模块职责
升级了 Java 版本	✅	§2 技术约束
换了构建命令或 profile	✅	§6 构建与测试
引入了新的框架或中间件	✅	§2 技术约束 + §4 架构原则
修了一个 bug	❌	—
加了几个接口	❌	—
重构了包命名	✅	§5 包命名规范

7.3 怎么更新

最简单的方式：直接在对话中告诉 AI。

```
请更新 project-context.md：
- 技术约束新增：引入了 Kafka 2.8 作为消息队列
- 构建命令新增：mvn clean package -P kafka -pl server/fs-cia-lis -am -DskipTests
```

AI 会自动把信息写到 `project-context.md` 的对应章节。

手动更新也可以：直接编辑 `project-context.md` 文件。

7.4 不要写到 CLAUDE.md 或 AGENTS.md

这两个文件是路由文件，不是知识库。 它们的作用是引导 AI 去读 `project-context.md` 。

如果你把项目信息写在 `CLAUDE.md` 里，会导致：

- `CLAUDE.md` 说一件事， `project-context.md` 说另一件事
- AI 不确定以哪个为准
- 不同平台（Claude/Codex/Qoder）看到不同的信息

所有项目知识 → `project-context.md` ，无例外。

---

8. 多 Story 并行开发

8.1 Story 隔离

每个 Story 有独立的目录和文档链：

```
iLink-doc/
├── kcia-1520/      ← Story A 的所有文档
│   ├── kcia-1520-需求定义.md
│   ├── kcia-1520-pm.master.md
│   └── ...
└── kcia-1521/      ← Story B 的所有文档，完全隔离
```

```
    ├── kcia-1521-需求定义.md
    └── ...
```

文档层面完全隔离，互不影响。

8.2 什么时候可以并行

场景	能否并行	原因
两个 Story 改不同模块的不同文件	✅ 可以	无代码冲突
两个 Story 改同一个模块的不同文件	⚠️ 小心	可能有编译依赖
两个 Story 改同一个文件	❌ 串行	会有 Git 冲突
两个 Story 都改数据库	⚠️ 小心	migration 顺序可能冲突

8.3 并行开发的建议

1. **先看 `[TASK_ALLOCATION]`**：两个 Story 的设计完成后，对比文件清单是否有交集
2. **无交集 → 放心并行**
3. **有交集 → 串行处理**：先完成并提交一个，再开始另一个
4. **不确定 → 问 AI**：在对话中描述两个 Story 的范围，让 AI 判断是否有冲突

8.4 不同人做不同 Story

同事 A 用 Claude Code 做 kcia-1520，同事 B 用 Qoder 做 kcia-1521——完全没问题。

两个人读的是同一份 `project-context.md`，产出到不同的 Story 目录，最终各自 `git commit`。

---

9. 常见问题排查

9.1 AI 说"找不到需求定义文件"

原因：你执行了 `/ilink-pm kcia-1520` 但没有先执行 `/ilink-init kcia-1520`。

解决：

```
/ilink-init kcia-1520
# 编辑需求定义后再执行
/ilink-pm kcia-1520
```

9.2 Designer 输出了 STAGING 但我觉得设计没问题

`STAGING` 是 Designer 的**默认状态**，所有设计都需要你审核后手动推进。这不是异常，而是 Human-Gate 机制的正常表现。

解决：审核设计后执行 `ilink-approve kcia-1520`。

9.3 PM 输出了 STAGING，有 `[待确认]` 项

解决：

```
# 推荐：使用 refine 逐条确认
/ilink-refine kcia-1520
# AI 会列出所有 [待确认] 项，你逐条给出决策
# 完成后执行 ilink-approve
```

```
# 或者：直接推进（承担风险）
ilink-approve kcia-1520
```

9.4 Coder 改了不在设计清单里的文件

查看 `code.master.md` 中的 `[DEVIATIONS]` 章节。Coder 如果偏离了设计，应该在这里说明理由。

判断：

- 理由合理（如发现设计遗漏了必要的依赖文件）→ 接受
- 理由不合理 → QA 通常会捕获这个问题；也可以手动撤销后重新执行

9.5 QA 一直 FAIL_BACK_TO_CODER，循环修不好

看 `ilink-status kcia-1520` 的回流次数：

- 1–2 次回流 → 正常，继续让 Coder 修
- 3 次 → 触发熔断，人工介入（见第 6.2 节）

9.6 project-context.md 内容过时了

```
/ilink-bootstrap
```

重新执行 Bootstrap 会**重新分析项目**并更新 `project-context.md`。不会覆盖你手动添加的内容，但会补充新发现的信息。

或者直接在对话中说："请更新 project-context.md 中的 §2 技术约束，Java 版本已从 8 升级到 17"。

9.7 我想跳过某个阶段

强烈不建议，但如果你确实需要：

想跳过	后果	替代方案
PM	Designer 没有业务边界，设计可能发散	PM 很快（30 秒），别跳
Designer	Coder 没有文件清单，可能乱改	自己写个简单的设计要点给 Coder
Human-Gate	设计未审核就编码，风险自担	快速扫一眼就 approve
QA	没有代码审查，bug 风险增加	QA 也很快（1–2 分钟），别跳

9.8 我想在非 iLink 模式下和 AI 对话

完全可以。iLink 不阻止你正常使用 AI。

执行了 Bootstrap 之后，AI 会读到 `CLAUDE.md` / `AGENTS.md` 中的 iLink 引导，但这只让 AI 知道项目知识在哪里。你可以正常问问题、让 AI 帮你改代码、debug 等等。

区别：非 iLink 模式下的修改不会有 Master Doc 记录，也不会经过链式审查。适合快速修复、探索性工作、或与 iLink 无关的任务。

---

10. 反模式——千万别这样做

❌ 反模式 1：需求写一句话就开工

```
## 1. 功能描述
优化登录
```

后果：PM 输出的业务合同范围模糊 → Designer 猜测设计方向 → Coder 写的代码可能不是你想要的 → 来回返工 3 轮。

正确做法：花 10 分钟写清楚功能范围和验收标准。

❌ 反模式 2：不审设计就 approve

```
/ilink-design kcia-1520
ilink-approve kcia-1520    # 看都不看就 approve
```

后果：设计方向如果有偏差，Coder 按错误方向写的代码，QA 按错误基线审查，全链路错。Coder 写的代码越多，浪费越大。

正确做法：至少看 `[TASK_ALLOCATION]` 的文件列表和 `[DESIGN_DECISIONS]`，确认方向正确。

❌ 反模式 3：在 Coder 阶段口头追加需求

```
"代码写得不错，顺便把登录日志也改一下吧"
```

后果：Coder 有白名单约束，追加的需求不在设计文件清单里。要么 Coder 拒绝（你觉得不灵活），要么 Coder 违反协议（QA 会标记偏差）。

正确做法：追加的需求开一个新 Story，或回到 Designer 阶段更新设计。

❌ 反模式 4：手动改了代码后继续走流水线

```
在 /ilink-coder 之后，你手动改了几个文件
然后执行 /ilink-qa
```

后果：QA 审查时对照的是 `code.master.md` 中的变更清单，你手动改的文件不在清单里。QA 可能漏审你的修改，也可能因为代码和 code.master.md 不一致而困惑。

正确做法：

- 小修改 → 在 QA 之后手动改，然后一起提交
- 大修改 → 回到 Coder 阶段重新执行，让 AI 把你的修改纳入变更清单

❌ 反模式 5：多人同时处理同一个 Story

```
同事 A 在执行 /ilink-design kcia-1520
同事 B 也在执行 /ilink-design kcia-1520
```

后果：两个 AI 实例同时写同一个 `design.master.md`，后写的覆盖先写的。

正确做法：一个 Story 在任一时刻只有一个人操作。

❌ 反模式 6：忽略 `[待确认]` 标记一路推进

```
PM 输出 STAGING，有 3 个 [待确认] 项
直接 ilink-approve，不看不管
Designer 基于猜测做设计
Coder 基于猜测写代码
最终发现猜错了，全部返工
```

后果：`[待确认]` 项原样传入下游，下游角色只能基于假设工作。假设错了，全链路返工。

正确做法：执行 `/ilink-refine` 逐条确认，花 5 分钟确认能省 2 小时返工。

附录 A：命令速查表

核心流水线命令

命令	阶段	谁执行	输出文件
`/ilink-init <story>`	初始化	人类	需求定义模板
`/ilink-pm <story>`	需求分析	AI	`<story>-pm.master.md`
`/ilink-design <story>`	技术设计	AI	`<story>-design.master.md`
`ilink-approve <story>`	审核推进	人类	（更新 Metadata Status）
`/ilink-coder <story>`	编码	AI	源码文件 + `<story>-code.master.md`
`/ilink-qa <story>`	审查	AI	`<story>-review.master.md`

辅助命令

命令	用途	谁执行
`/ilink-refine <story>`	修订 STAGING 文档（逐条确认阻塞项）	AI + 人类
`ilink-status [story]`	查看流水线状态与下一步建议	人类
`/ilink-bootstrap`	项目冷启动（生成项目知识库）	AI

注： `/ilink-*` 是 AI 执行的 slash command，`ilink-*`（无斜杠）是 shell 脚本。

附录 B：文件地图

```
<project_root>/
├── project-context.md              ← 项目知识库（AI 每次都读）
├── CLAUDE.md                       ← Claude 入口路由
├── AGENTS.md                       ← Codex/其他 AI 入口路由
│
├── iLink/                          ← 框架文件（不要修改）
│   ├── iLink-root-spec.md              ← 根规范
│   ├── iLink-implementation-guide.md   ← 实施手册
│   └── souls/                          ← 角色规范
│       ├── universal.soul.md
│       ├── pm.soul.md
│       ├── design.soul.md
│       ├── coder.soul.md
│       └── qa.soul.md
│
├── iLink-doc/                      ← Story 文档（提交到 Git）
│   ├── kcia-1520/
│   │   ├── kcia-1520-需求定义.md       ← 你写的
│   │   ├── kcia-1520-pm.master.md      ← AI 写的
│   │   ├── kcia-1520-design.master.md  ← AI 写的，你审核的
│   │   ├── kcia-1520-code.master.md    ← AI 写的
│   │   └── kcia-1520-review.master.md  ← AI 写的
│   └── kcia-1521/
│       └── ...
│
├── .claude/commands/               ← Claude CLI 命令
├── .codex/commands/                ← Codex CLI 命令
```

```
├── .qoder/commands/                ← Qoder CLI 命令
└── src/                            ← 源代码
```

提交规则：

- ✅ 提交：`iLink-doc/`、`project-context.md`、`CLAUDE.md`、`AGENTS.md`、`iLink/`、源码
- ❌ 不提交：`.retry_count`（信号文件）

附录 C：状态流转图

```
                    ┌──────────────────┐
                    │   /ilink-init    │
                    │ (人类创建Story)  │
                    └──────────────────┘
                             │
                             ▼
                    ┌──────────────────┐
                    │ 编辑需求定义     │
                    │ (人类手写)       │
                    └──────────────────┘
                             │
                             ▼
                    ┌──────────────────┐
                    │   /ilink-pm      │      STAGING
                    │ (AI 需求分析)    │──────→ /ilink-refine 确认
                    └──────────────────┘      → ilink-approve 推进
                             │
                    PENDING_DESIGNER
                             │
                             ▼
                    ┌──────────────────┐
                    │ /ilink-design    │
                    │ (AI 技术设计)    │
                    └──────────────────┘
                             │
                        STAGING ← 默认
                             │
                             ▼
⛔                  ┌──────────────────┐
                    │ 人类审核设计     │
                    │ /ilink-refine    │ ← 可选：修订设计
                    │ ilink-approve    │ ← 推进
                    └──────────────────┘
                             │
                    PENDING_CODER
                             │
                             ▼
                    ┌──────────────────┐
                    │ /ilink-coder     │
                    │ (AI 编码)        │
                    └──────────────────┘
                             │
                        PENDING_QA
                             │
                             ▼
```

```
                    ┌──────────────────┐
                    │   /ilink-qa      │
                    │  (AI 代码审查)    │
                    └──────────────────┘
                             │
                             │
          ┌──────────────────┼──────────────────┐
          │                  │                  │
          ▼                  ▼                  ▼
     COMPLETED        FAIL_BACK_TO        STAGING
      (通过)           _CODER             (上游问题)
        │              (代码问题)            │
        │                  │                 │
        ▼                  ▼                 ▼
  git commit        /ilink-coder      /ilink-refine
                     (回流修复)        (讨论根因)
                         │
                         ▼
                    /ilink-qa
                     (重新审查)
                         │
                    ≥3次回流?
                    ├─ 否 → 继续
                    └─ 是 → ⚠️ 熔断，人工介入
```